Diffusion-based Task Offloading, Cooperative Service Caching, and NOMA Resource Allocation in Jammer-Aware Multi-UAV MEC-IoT Networks

Nayanjit Talukdar, Tivranshu, Kushagra Gupta, Shivam Chaturvedi, Tanmay Mathurvaishya, Nabajyoti Mazumdar, *Senior Member, IEEE*

***Abstract*—UAV-assisted Mobile Edge Computing (MEC) has emerged as a promising paradigm for low-latency IoT services in dynamic environments. However, jointly optimizing task offloading, cooperative service caching, NOMA-based communication, and UAV trajectory under adversarial jamming remains challenging due to hybrid decision variables, strong interdependencies, and high-dimensional combinatorial action spaces that existing approaches fail to handle within a unified framework. To address this, we propose the *Diffusion-based Multi-UAV Joint Optimization (DMJO)* framework, formulated as a stochastic MINLP and solved via a two-timescale hierarchical MDP. The proposed framework considers a three-tier service availability architecture, where WDs and UAVs cache limited service programs, while the BS maintains the complete service library and provides cache-miss service retrieval without participating in task computation. At the macro-timescale, MAPPO jointly optimizes UAV trajectories and cooperative caching via an adaptive priority-based strategy. At the micro-timescale, a dual-head diffusion-enhanced actor combines a Gaussian policy head for stable PPO-based optimization with a denoising diffusion head for structured exploration of combinatorial offloading spaces. Analytical NOMA power allocation via bisection decouples communication from RL policy updates. The framework operates under a CTDE paradigm, ensuring scalable real-time deployment. Simulation results demonstrate that DMJO significantly outperforms state-of-the-art baselines in jointly optimizing task completion latency and UAV energy consumption, while maintaining robustness under adversarial jamming.**

***Index Terms*—Unmanned aerial vehicles, mobile edge computing, internet of things, diffusion deep reinforcement learning, NOMA, task offloading, cooperative caching, jammer-aware networks.**

I. Introduction

THE evolution of 5G/6G wireless networks is enabling massive Internet of Things (IoT) deployments for applications such as smart cities, industrial automation, and disaster response, which demand ultra-reliable and low-latency communication [1]. However, the limited computation and energy capabilities of IoT devices (hereafter referred to as wireless devices, WDs) make it challenging to process increasingly complex and data-intensive tasks locally. Unmanned aerial vehicles (UAVs), equipped with mobile edge computing (MEC) capabilities, have emerged as a promising solution due to their flexibility, rapid deployment, and ability to provide on-demand communication and computation services in dynamic environments [2], [3].

Despite the growing adoption of UAV-assisted MEC, existing system designs remain fundamentally limited by fragmented optimization strategies that treat task offloading, caching, communication, and mobility as loosely coupled components [4]. In practical IoT deployments, however, these elements are inherently interdependent. Service caching determines whether a task can be executed locally at the WD, processed by a serving UAV with locally cached services, or served after retrieving the required service from another UAV or the Base station (BS). In the considered architecture, service availability is supported through a three-tier structure: resource-constrained WDs cache a limited number of services for local execution, UAVs store popular service programs and cooperatively exchange missing services through air-to-air (A2A) links, whereas the BS maintains the complete service library and acts only as a service repository for cache-miss recovery. The BS does not participate in computation task execution; instead, it supplies missing service programs to UAVs when the requested service is unavailable in the WD and cooperative UAV caches. Therefore, caching directly affects service-fetching delay and offloading efficiency; offloading decisions reshape traffic patterns and interference in NOMA-based communication; and UAV mobility dynamically alters channel conditions while interacting with adversarial jamming environments. This tight coupling creates a set of challenges that existing approaches have not simultaneously addressed. While the primary objective in such systems is latency minimization, ensuring energy-efficient operation under UAV energy constraints remains equally critical.

Specifically, three critical gaps remain open in the literature. *First*, most task offloading frameworks [5], [6] treat caching as a static background condition, ignoring the dynamic interplay between service availability and offloading feasibility across epochs. *Second*, while two-timescale frameworks such as [7] decouple slow and fast-varying decisions to reduce complexity, they rely on heuristic approximations and standard MLP-based policies that fail to explore high-dimensional combinatorial offloading action spaces effectively. *Third*, the simultaneous presence of adversarial jammers, hybrid discrete-continuous decision variables, and multi-UAV coordination introduces a level of stochasticity and coupling that neither convex opti-

(Corresponding author: Nabajyoti Mazumdar.)

N. Talukdar, Tivranshu, K. Gupta, S. Chaturvedi, T. Mathurvaishya and N. Mazumdar are with the Department of Information Technology, Indian Institute of Information Technology, Allahabad. India. (e-mail: rsi2023502@iiita.ac.in, pml2025002@iiita.ac.in, iit2023133@iiita.ac.in, iit2023126@iiita.ac.in, iit2023117@iiita.ac.in, nabajyoti@iiita.ac.in).

mization [8] nor standard single-agent reinforcement learning [5] can tractably handle within a unified framework.

To address these limitations simultaneously, we propose a *Diffusion-based Multi-UAV Joint Optimization (DMJO)* framework. Unlike [7], which decouples decisions through heuristic approximations, DMJO preserves strong inter-component dependencies through a hierarchical Markov decision process (HMDP) formulation. Unlike standard DRL approaches [3], DMJO replaces the conventional MLP actor with a dual-head diffusion architecture that captures multi-modal action distributions, enabling structured exploration of combinatorial offloading spaces. Furthermore, unlike works that optimize caching and communication independently [9], [10], DMJO jointly optimizes adaptive cooperative caching, analytical NOMA power allocation, and MAPPO-based trajectory control within a single centralized training with decentralized execution (CTDE) paradigm, while explicitly incorporating UAV energy consumption into the optimization objective. The main contributions of this paper are as follows:

- We formulate a jammer-aware multi-UAV MEC-IoT optimization problem as a stochastic MINLP that jointly models task offloading, three-tier service availability with WD caching, cooperative UAV caching, and BS-assisted service retrieval, NOMA-based communication, 3D UAV trajectory control, and UAV energy constraints.
- We develop a two-timescale hierarchical MDP framework for joint decision-making, where macro-timescale decisions optimize UAV trajectory and cooperative caching, while micro-timescale decisions handle task offloading and NOMA-based communication under dynamic task arrivals and jammer-induced interference.
- We propose a diffusion-enhanced learning architecture for combinatorial task offloading, where a dual-head actor combines a PPO-compatible Gaussian policy head with a denoising diffusion head to enable stable policy optimization and structured exploration of high-dimensional offloading actions.
- We integrate MAPPO-based multi-UAV trajectory optimization and analytical NOMA power allocation under a CTDE paradigm, enabling scalable coordination among UAVs while reducing RL action dimensionality. Extensive simulations validate that DMJO reduces task completion latency and UAV energy consumption while maintaining robustness under adversarial jamming.

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the system model. Section IV describes the proposed methodology. Section V presents simulation results, and Section VI concludes the paper.

II. Related Work

Task offloading in UAV-enabled MEC has been extensively studied to reduce latency and energy consumption. Early works employed convex optimization and game-theoretic approaches for joint offloading and resource allocation [8], while recent studies have leveraged DRL to capture dynamic system interactions [5]. Nguyen et al. [11] studied dependency-aware task offloading in collaborative UAV-MEC networks, using a discrete whale optimization algorithm with convex resource allocation to minimize latency for DAG-based dependent subtasks under UAV energy constraints. Multi-agent learning frameworks have further enabled distributed decision-making across UAVs [3], [6]. However, these approaches focus primarily on offloading in isolation, treating caching as a static precondition and ignoring its dynamic coupling with offloading feasibility, NOMA interference, and adversarial jamming effects.

Cooperative caching and NOMA-based communication have been explored to improve service availability and spectral efficiency. Araf et al. [12] proposed a multi-agent actor-critic framework for UAV-assisted cooperative caching, where UAVs and ground base stations jointly learn dynamic caching policies to improve global cache hit ratio. Xu *et al.* [9] proposed a cooperative caching framework enhancing service availability through inter-node collaboration, while hierarchical and learning-based caching strategies have also been investigated [13], [14]. Wang et al. [15] proposed a cache-aided MEC framework for IoT networks, using deep graph RL to jointly optimize computation offloading and communication-computing-caching resource allocation for reducing system latency in large-scale dynamic MEC environments. In [16] the authors proposed a multi-agent two-timescale DRL framework for UAV-assisted coded caching, jointly optimizing slow-timescale content placement and fast-timescale power allocation to improve content hit ratio and transmission delay in emergency networks. The authors in [17] proposed CACHE-6G, a MADRL-based UAV-UGV cooperative caching and energy management framework that integrates RIS and WPT to optimize mobility, cache placement, and energy replenishment in 6G networks. For NOMA, joint trajectory and resource allocation have been studied in UAV-assisted networks [1], [10]. In [18], the authors proposed a C-NOMA-enabled space-air-ground edge computing framework, where task offloading, UAV trajectory, program caching, routing, subchannel allocation, and power control are jointly optimized to reduce system delay. Despite these advances, caching and communication are typically optimized independently, without integrating their interaction with task offloading decisions or UAV mobility.

To address heterogeneous dynamics and security threats, two-timescale frameworks and advanced learning approaches have gained traction. Sun *et al.* [7] decoupled slow and fast-varying decisions to reduce complexity, but rely on heuristic approximations that struggle to scale to tightly coupled hybrid decision spaces. Anti-jamming trajectory and power control have been studied separately [19], [20], but without integration into a joint MEC optimization framework. More recently, diffusion models have shown strong capability in modeling high-dimensional decision spaces [21], [22], yet their application to joint task offloading, caching, and trajectory optimization in jammer-aware multi-UAV systems remains unexplored. In contrast, the proposed DMJO framework is the first to unify diffusion-enhanced offloading, cooperative caching, analytical NOMA power allocation, and MAPPO-based trajectory control within a single hierarchical architecture under adversarial conditions.

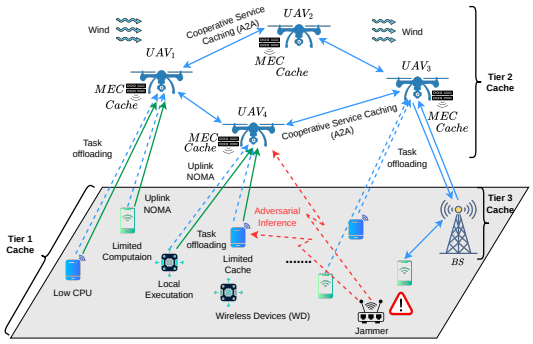


Fig. 1. Proposed Multi UAV MEC Network

III. System Model and Problem Formulation

A. Network Model

In Fig 1, we consider a UAV-enabled MEC-assisted IoT network operating over a finite horizon divided into K decision epochs, indexed by $k \in \mathcal{K} \triangleq \{1, 2, \ldots, K\}$. The network consists of a set of mobile WDs, a set of MEC-enabled UAVs, one terrestrial BS, and a set of jammers. Let $\mathcal{N} \triangleq \{1, 2, \ldots, N\}$ denote the set of WDs. The position of WD $n \in \mathcal{N}$ at epoch k is denoted by $\mathbf{w}_n(k) = [x_n(k), y_n(k), 0]^\top$. Each WD has a local CPU frequency f_n^{loc} and limited cache capacity C_n, and may either execute a generated task locally or offload it to a UAV. Let $\mathcal{M} \triangleq \{1, 2, \ldots, M\}$ denote the set of UAVs. UAV $m \in \mathcal{M}$ flies in 3D with position

$$\mathbf{q}_m(k) = [x_m(k), y_m(k), h_m(k)]^\top, \quad h_m(k) \in [h_{\min}, h_{\max}], \tag{1}$$

where $h_{\min}$ and $h_{\max}$ are the minimum and maximum allowable flight altitudes, respectively. Each UAV is equipped with an onboard MEC server of CPU frequency f_m^{uav} and cache capacity C_m^{uav}. UAVs execute offloaded tasks and cooperatively exchange cached service programs through A2A links when the required service is unavailable at the serving UAV. A single BS is deployed at $\mathbf{b} = [x_b, y_b, 0]^\top$ and acts as a control anchor and complete service repository. It provides missing service programs to UAVs when the required service is unavailable in the serving UAV and cooperative UAV caches, but it does not participate in task computation. Thus, tasks are executed only at the originating WD or at a serving UAV, while the BS supports cache-miss service retrieval. The resulting service access structure consists of WD-level local caching, UAV-level cooperative caching, and BS-assisted service retrieval. Let $\mathcal{J} \triangleq \{1, 2, \ldots, J\}$ denote the set of jammers. Jammer $j \in \mathcal{J}$ is located at $\mathbf{u}_j(k) = [x_j(k), y_j(k), 0]^\top$ and transmits intentional interference to degrade WD-to-UAV uplink offloading links. The system evolves in epochs of duration Δt, and the mobility of WDs and UAVs is captured through $\{\mathbf{w}_n(k)\}$ and $\{\mathbf{q}_m(k)\}$.

B. Mobility Model of WDs

At the beginning of the mission horizon, all WDs are uniformly distributed within the monitoring region $\mathcal{A}$. The mobility of each WD $n \in \mathcal{N}$ is modeled using the Gauss-Markov (GM) model [23], which captures temporal correlation in both speed and heading direction. At each epoch $k \in \mathcal{K}$, the speed $v_n(k)$ and heading angle $\theta_n(k)$ evolve as

$$v_n(k) = \rho_v v_n(k-1) + (1-\rho_v)\bar{v}_n + \sqrt{1-\rho_v^2}\,\xi_n^v(k), \tag{2}$$

$$\theta_n(k) = \rho_\theta \theta_n(k-1) + (1-\rho_\theta)\bar{\theta}_n + \sqrt{1-\rho_\theta^2}\,\xi_n^\theta(k), \tag{3}$$

where $\bar{v}_n$ and $\bar{\theta}_n$ denote the mean speed and heading direction of WD n, respectively. The parameters $\rho_v, \rho_\theta \in [0, 1]$ control the memory of the mobility process, while $\xi_n^v(k) \sim \mathcal{N}(0, \sigma_v^2)$ and $\xi_n^\theta(k) \sim \mathcal{N}(0, \sigma_\theta^2)$ are independent Gaussian perturbations. Based on the speed and heading at epoch $k-1$, the horizontal position of WD n is updated as

$$x_n(k) = x_n(k-1) + \Delta t\, v_n(k-1)\cos\theta_n(k-1), \tag{4}$$

$$y_n(k) = y_n(k-1) + \Delta t\, v_n(k-1)\sin\theta_n(k-1), \tag{5}$$

such that $\mathbf{w}_n(k) = [x_n(k), y_n(k), 0]^\top$. If the updated position moves outside $\mathcal{A}$, a specular reflection rule is applied to the heading direction to keep the WD trajectory within the monitoring region.

C. Communication Model

1) Air-to-Ground Channel Model: The primary offloading links between WDs and UAVs are modeled as probabilistic air-to-ground (A2G) channels, wherein the instantaneous propagation condition is either line-of-sight (LoS) or non-line-of-sight (NLoS) depending on the relative geometry and the surrounding environment [24]. For WD $n \in \mathcal{N}$ and UAV $m \in \mathcal{M}$ at epoch k, the Euclidean distance is given by

$$\begin{aligned} d_{n,m}(k) &= \|\mathbf{q}_m(k) - \mathbf{w}_n(k)\|_2 \\ &= \sqrt{\big(x_m(k) - x_n(k)\big)^2 + \big(y_m(k) - y_n(k)\big)^2 + h_m^2(k)}, \end{aligned} \tag{6}$$

and the corresponding elevation angle (in degrees) is $\phi_{n,m}(k) = \frac{180}{\pi}\arcsin\!\left(\frac{h_m(k)}{d_{n,m}(k)}\right)$. The probability that the link between WN n and UAV m is LoS at epoch k is expressed as

$$\mathcal{P}_{n,m}^{\text{LoS}}(k) = \frac{1}{1 + a\exp\big(-b\big(\phi_{n,m}(k) - a\big)\big)}, \tag{7}$$

where a and b are environment-dependent constants determined by the propagation environment. The NLoS probability is then $\mathcal{P}_{n,m}^{\text{NLoS}}(k) = 1 - \mathcal{P}_{n,m}^{\text{LoS}}(k)$.

Let β_{LoS} and β_{NLoS} denote the large-scale channel gain coefficients under LoS and NLoS propagation, respectively, where $\beta_{\text{LoS}} > \beta_{\text{NLoS}} > 0$. Then, the corresponding LoS and NLoS channel gains are modeled as

$$g_{n,m}^{\text{LoS}}(k) = \beta_{\text{LoS}}\big(d_{n,m}(k)\big)^{-\ell}, \tag{8}$$

$$g_{n,m}^{\text{NLoS}}(k) = \beta_{\text{NLoS}}\big(d_{n,m}(k)\big)^{-\ell}, \tag{9}$$

where ℓ denotes the path-loss exponent. Using the probability-weighted average large-scale channel model, the effective A2G channel gain between WN n and UAV m at epoch k is given by

$$\begin{aligned} g_{n,m}(k) &= \mathcal{P}_{n,m}^{\text{LoS}}(k)\, g_{n,m}^{\text{LoS}}(k) + \mathcal{P}_{n,m}^{\text{NLoS}}(k)\, g_{n,m}^{\text{NLoS}}(k) \\ &= \Big[\mathcal{P}_{n,m}^{\text{LoS}}(k)\beta_{\text{LoS}} + \mathcal{P}_{n,m}^{\text{NLoS}}(k)\beta_{\text{NLoS}}\Big]\big(d_{n,m}(k)\big)^{-\ell}. \end{aligned} \tag{10}$$

2) Air-to-Air Channel Model: Inter-UAV links are used only for cooperative caching exchange, where one UAV may fetch a required service or cached content from another UAV. Due to the relatively high operating altitude of UAVs and the absence of significant aerial obstructions, inter-UAV links are assumed to experience dominant LoS propagation. Therefore, the A2A channel gain between UAV m and UAV $m' \in \mathcal{M} \setminus \{m\}$ at epoch k is modeled as [25],

$$g_{m,m'}(k) = \beta_{\mathrm{LoS}} \big(d_{m,m'}(k)\big)^{-\ell}, \tag{11}$$

where $d_{m,m'}(k) = \|\mathbf{q}_m(k) - \mathbf{q}_{m'}(k)\|_2$ denotes the Euclidean distance between UAV m and UAV m'.

3) Jammer Interference Model: Each jammer transmits fixed-power interference over the WD-to-UAV uplink spectrum. Since task offloading is the main wireless bottleneck, jamming is assumed to affect only WD-to-UAV links. For jammer $j \in \mathcal{J}$ and UAV m, the jammer-to-UAV distance is, $d_{j,m}(k) = \|\mathbf{q}_m(k) - \mathbf{u}_j(k)\|_2$. The elevation angle is, $\phi_{j,m}(k) = \frac{180}{\pi} \arcsin\Big(\frac{h_m(k)}{d_{j,m}(k)}\Big)$. Using the same probabilistic A2G model, the jammer-to-UAV channel gain is

$$g_{j,m}(k) = \Big[\mathcal{P}^{\mathrm{LoS}}_{j,m}(k)\beta_{\mathrm{LoS}} + \mathcal{P}^{\mathrm{NLoS}}_{j,m}(k)\beta_{\mathrm{NLoS}}\Big]\big(d_{j,m}(k)\big)^{-\ell}, \tag{12}$$

where

$$\mathcal{P}^{\mathrm{LoS}}_{j,m}(k) = \frac{1}{1 + a\exp\big(-b\big(\phi_{j,m}(k) - a\big)\big)} \tag{13}$$

and $\mathcal{P}^{\mathrm{NLoS}}_{j,m}(k) = 1 - \mathcal{P}^{\mathrm{LoS}}_{j,m}(k)$. Therefore, the aggregate jamming interference received at UAV m is, $I_m(k) = \sum_{j\in\mathcal{J}} P^{\mathrm{jam}}_j\, g_{j,m}(k)$, where P^{jam}_j is the transmit power of jammer j.

4) NOMA-Based Uplink Transmission: To enable simultaneous task offloading from multiple wireless nodes to the same UAV, power-domain NOMA is adopted for the wireless node-to-UAV uplink [2]. Let $a_{n,m}(k) \in \{0,1\}$ denote the association indicator, where $a_{n,m}(k) = 1$ means that wireless node n offloads its task to UAV m at epoch k, and $a_{n,m}(k) = 0$ otherwise. Each wireless node can associate with at most one UAV at each epoch, i.e., $\sum_{m\in\mathcal{M}} a_{n,m}(k) \le 1, \quad \forall n \in \mathcal{N},\ \forall k \in \mathcal{K}$. Let $\mathcal{N}_m(k) \triangleq \{n \in \mathcal{N} : a_{n,m}(k) = 1\}$ denote the NOMA cluster served by UAV m at epoch k. To ensure practical SIC complexity, the number of simultaneously served wireless nodes at each UAV is limited by $|\mathcal{N}_m(k)| \le N^{\max}_m, \quad \forall m \in \mathcal{M},\ \forall k \in \mathcal{K}$, where $N^{\max}_m$ is the maximum allowable NOMA cluster size of UAV m.

Let $p_n(k)$ denote the uplink transmit power of wireless node n at epoch k. Under uplink NOMA, UAV m employs successive interference cancellation (SIC) to decode the superimposed signals from all wireless nodes in $\mathcal{N}_m(k)$. Assuming ideal SIC, previously decoded signals are perfectly cancelled. The decoding order is determined by the ascending order of effective channel gains, i.e., from the weakest to the strongest channel. Let $\pi_m(k)$ denote the permutation over $\mathcal{N}_m(k)$ such that, $g_{\pi^{(1)}_m(k),m}(k) \le g_{\pi^{(2)}_m(k),m}(k) \le \cdots \le g_{\pi^{(|\mathcal{N}_m(k)|)}_m(k),m}(k)$, where $\pi^{(i)}_m(k)$ is the WD decoded at the i-th SIC stage by UAV m. For a WD $n = \pi^{(i)}_m(k)$, the signals of nodes decoded in stages $1,\ldots,i-1$ have already been removed, while the signals of nodes decoded in stages $i+1,\ldots,|\mathcal{N}_m(k)|$ remain as intra-cluster interference. Therefore, the received SINR of node $n = \pi^{(i)}_m(k)$ at UAV m is given by

$$\gamma_{n,m}(k) = \frac{p_n(k) g_{n,m}(k)}{\sum_{r=i+1}^{|\mathcal{N}_m(k)|} p_{\pi^{(r)}_m(k)}(k) g_{\pi^{(r)}_m(k),m}(k) + I_m(k) + \sigma_0^2}. \tag{14}$$

where σ_0^2 denotes the additive white Gaussian noise power at the UAV receiver. Since different UAVs are assumed to operate on orthogonal resource blocks, inter-cluster uplink interference among different UAVs is ignored. Hence, the achievable uplink transmission rate of wireless node n offloading to UAV m at epoch k is

$$R_{n,m}(k) = a_{n,m}(k)\, B \log_2\big(1 + \gamma_{n,m}(k)\big), \tag{15}$$

where B denotes the shared uplink bandwidth allocated to the NOMA cluster served by UAV m. To guarantee offloading reliability, the achievable uplink rate must satisfy the minimum QoS requirement of each wireless node, $R_{n,m}(k) \ge a_{n,m}(k)\, R^{\min}_n, \quad \forall n \in \mathcal{N},\ \forall m \in \mathcal{M},\ \forall k \in \mathcal{K}$. Moreover, the uplink transmit power of each wireless node is constrained by, $0 \le p_n(k) \le P^{\max}_n, \quad \forall n \in \mathcal{N},\ \forall k \in \mathcal{K}$.

D. Three-Tier Service Caching and Availability Model

1) Service Library and Popularity: Let $\mathcal{L} \triangleq \{1,2,\ldots,L\}$ denote the service library, where each service $l \in \mathcal{L}$ represents an application program required for task execution and occupies storage size s_l. Since service requests in MEC systems often follow a long-tail distribution, the request probability of service l is modeled using a Zipf distribution, $p_l = \frac{l^{-\zeta}}{\sum_{i=1}^{L} i^{-\zeta}}, \quad l \in \mathcal{L}$, where services are indexed in descending order of popularity and $\zeta \ge 0$ controls the skewness of the request distribution [8].

2) Three-Tier Service Caching States: To reduce service access latency and improve execution flexibility, the proposed framework adopts a three-tier service availability architecture. At the first tier, WDs cache a limited number of service programs for local task execution. At the second tier, UAVs cache popular service programs and cooperatively share missing services through A2A links. At the third tier, the BS maintains the complete service library and provides missing service programs to UAVs when the requested service is unavailable in the cooperative UAV cache layer. The BS is not used for task computation; it only supports cache-miss service retrieval. Let $c_{n,l}(k) \in \{0,1\}$ and $c_{m,l}(k) \in \{0,1\}$ denote the caching state of service l at WD n and UAV m during epoch k, respectively. These variables are defined as

$$c_{n,l}(k) = \begin{cases} 1, & \text{if service } l \text{ is cached at WD } n \text{ during epoch } k, \\ 0, & \text{otherwise,} \end{cases} \tag{16}$$

and

$$c_{m,l}(k) = \begin{cases} 1, & \text{if service } l \text{ is cached at UAV } m \text{ during epoch } k, \\ 0, & \text{otherwise.} \end{cases} \tag{17}$$

Since WDs and UAVs have limited storage capacity, their caching decisions must satisfy $\sum_{l\in\mathcal{L}} c_{n,l}(k) s_l \le C_n, \quad \forall n \in \mathcal{N},\ \forall k \in \mathcal{K}$, and $\sum_{l\in\mathcal{L}} c_{m,l}(k) s_l \le C^{\mathrm{uav}}_m, \quad \forall m \in \mathcal{M},\ \forall k \in \mathcal{K}$, where C_n and C^{uav}_m denote the cache capacities of WD n

and UAV m, respectively. In general, $C_n \ll C_m^{\mathrm{uav}}$, reflecting the fact that WDs are resource-constrained, whereas UAVs are equipped with relatively larger onboard storage.

In addition, the BS is modeled as a complete service repository. Let $c_{b,l}$ denote the availability of service l at the BS. Since the BS stores all services in the library, we have, $c_{b,l} = 1, \quad \forall l \in \mathcal{L}$. Unlike WDs and UAVs, the BS is assumed to have sufficient storage for the complete service library and is therefore not subject to a cache capacity constraint. However, retrieving a service from the BS incurs an additional BS-to-UAV service fetching delay.

3) Cooperative Inter-UAV Service Fetching: Due to finite UAV cache capacity, UAVs cooperatively share cached services through dedicated A2A links. If UAV m serves WD n but does not cache the required service $s_n(k)$, it may fetch the service from another UAV $m' \in \mathcal{M} \setminus \{m\}$ with $c_{m',s_n(k)}(k) = 1$. These A2A links are used only for service fetching, not for relaying WD task data. The cooperative service fetch rate is

$$R_{m,m'}^{\mathrm{coop}}(k) = B^{\mathrm{A2A}} \log_2\left(1 + \frac{P_{m'}^{\mathrm{uav}} g_{m,m'}(k)}{\sigma_0^2}\right), \tag{18}$$

where B^{A2A}, $P_{m'}^{\mathrm{uav}}$, and $g_{m,m'}(k)$ denote the A2A bandwidth, transmitting UAV power, and A2A channel gain, respectively.

4) BS-Assisted Service Fetching: If $s_n(k)$ is unavailable in the cooperative UAV cache layer, UAV m retrieves it from the BS repository. The BS-to-UAV fetching rate is

$$R_{b,m}^{\mathrm{BS}}(k) = B^{\mathrm{BS}} \log_2\left(1 + \frac{P_b g_{b,m}(k)}{\sigma_0^2}\right), \tag{19}$$

where B^{BS}, P_b, and $g_{b,m}(k)$ denote the BS-to-UAV bandwidth, BS transmit power, and BS-to-UAV channel gain, respectively. Let $\mathcal{M}_{m,l}^{\mathrm{coop}}(k) = \{m' \in \mathcal{M} \setminus \{m\} \mid c_{m',l}(k) = 1\}$ denote the set of UAVs caching service l. If $\mathcal{M}_{m,s_n(k)}^{\mathrm{coop}}(k) \neq \emptyset$, the service-providing UAV is selected as $m^\star = \arg\max_{m' \in \mathcal{M}_{m,s_n(k)}^{\mathrm{coop}}(k)} R_{m,m'}^{\mathrm{coop}}(k)$. Therefore, the service fetching delay is

$$T_{n,m}^{\mathrm{fetch}}(k) = \begin{cases} 0, & c_{m,s_n(k)}(k) = 1, \\ \dfrac{s_{s_n(k)}}{R_{m,m^\star}^{\mathrm{coop}}(k)}, & c_{m,s_n(k)}(k) = 0,\ \mathcal{M}_{m,s_n(k)}^{\mathrm{coop}}(k) \neq \emptyset, \\ \dfrac{s_{s_n(k)}}{R_{b,m}^{\mathrm{BS}}(k)}, & c_{m,s_n(k)}(k) = 0,\ \mathcal{M}_{m,s_n(k)}^{\mathrm{coop}}(k) = \emptyset. \end{cases} \tag{20}$$

Thus, $T_{n,m}^{\mathrm{fetch}}(k)$ captures local UAV cache hits, A2A cooperative fetching, and BS-assisted cache-miss recovery.

5) Effective Service Availability: The cooperative UAV-level service availability indicator is defined as

$$\tilde{c}_{m,l}^{\mathrm{uav}}(k) = \begin{cases} 1, & c_{m,l}(k) = 1, \\ 1, & \exists m' \in \mathcal{M} \setminus \{m\} : c_{m',l}(k) = 1, \\ 0, & \text{otherwise.} \end{cases} \tag{21}$$

Since the BS stores the complete service library, system-level service availability is $\tilde{c}_{m,l}^{\mathrm{sys}}(k) = 1, \quad \forall m \in \mathcal{M},\ \forall l \in \mathcal{L},\ \forall k \in \mathcal{K}$. Here, $\tilde{c}_{m,l}^{\mathrm{uav}}(k)$ indicates low-latency UAV-level availability, whereas $\tilde{c}_{m,l}^{\mathrm{sys}}(k)$ guarantees service retrieval through BS-assisted fetching.

6) Caching-Execution Feasibility: Local execution at WD n is feasible only when the required service is cached locally: $x_n^{\mathrm{loc}}(k) \leq c_{n,s_n(k)}(k),\ \forall n, k$. For UAV-assisted execution, a UAV cache miss does not block offloading because the service can be retrieved through A2A fetching or from the BS repository. Hence, UAV offloading feasibility is governed by task assignment, NOMA cluster-size, QoS, and UAV resource constraints, while caching affects only $T_{n,m}^{\mathrm{fetch}}(k)$.

E. Computation Model

1) Task Generation Model: At each epoch $k \in \mathcal{K}$, WD $n \in \mathcal{N}$ generates a task with probability p_0. Let $\delta_n(k) \in \{0,1\}$ denote the task-generation indicator. A generated task is represented as $T_n(k) = \big(D_n(k), F_n(k), s_n(k)\big)$, where $D_n(k)$ is the input data size, $F_n(k)$ is the required CPU cycles, and $s_n(k) \in \mathcal{L}$ is the required service type drawn according to Zipf popularity distribution (III-D1).

2) Execution Assignment: Each generated task is executed either locally at its originating WD or remotely at one serving UAV; the BS only provides missing services and does not execute tasks. Let $x_n^{\mathrm{loc}}(k) \in \{0,1\}$ and $a_{n,m}(k) \in \{0,1\}$ denote the local execution and UAV offloading indicators, respectively. The assignment constraint is

$$x_n^{\mathrm{loc}}(k) + \sum_{m \in \mathcal{M}} a_{n,m}(k) = \delta_n(k), \quad \forall n \in \mathcal{N},\ \forall k \in \mathcal{K}. \tag{22}$$

Local execution must satisfy (III-D6), while UAV execution remains feasible under cache misses because the required service can be retrieved through A2A fetching or BS-assisted retrieval.

3) Local Computation Delay: If $x_n^{\mathrm{loc}}(k) = 1$, task $T_n(k)$ is executed at WD n with CPU frequency f_n^{loc}, and the local delay is

$$T_n^{\mathrm{loc}}(k) = \frac{F_n(k)}{f_n^{\mathrm{loc}}}, \quad \forall n \in \mathcal{N},\ \forall k \in \mathcal{K}. \tag{23}$$

4) UAV Offloading Delay: If $a_{n,m}(k) = 1$, task $T_n(k)$ is offloaded to UAV m, and the completion delay includes uplink transmission, service fetching, queue waiting, and UAV computation delays. The uplink transmission delay is

$$T_{n,m}^{\mathrm{tx}}(k) = \frac{D_n(k)}{R_{n,m}(k)}. \tag{24}$$

The service fetching delay $T_{n,m}^{\mathrm{fetch}}(k)$ follows the three-tier availability model in (20). Each UAV maintains a FIFO computation queue. Let $Z_m(k)$ denote the queued workload at UAV m in CPU cycles. Its evolution is

$$Z_m(k{+}1) = \max\left\{ Z_m(k) + \sum_{n \in \mathcal{N}} a_{n,m}(k)\delta_n(k)F_n(k) - f_m^{\mathrm{uav}}\Delta t,\ 0 \right\}. \tag{25}$$

The corresponding queue waiting delay is

$$Q_m(k) = \frac{Z_m(k)}{f_m^{\mathrm{uav}}}. \tag{26}$$

After data and service availability, UAV m executes the task with computation delay

$$T_{n,m}^{\mathrm{cmp}}(k) = \frac{F_n(k)}{f_m^{\mathrm{uav}}}. \tag{27}$$

Thus, the total UAV-side task completion delay is

$$T_{n,m}^{\mathrm{uav}}(k) = T_{n,m}^{\mathrm{tx}}(k) + T_{n,m}^{\mathrm{fetch}}(k) + Q_m(k) + T_{n,m}^{\mathrm{cmp}}(k). \tag{28}$$

5) Unified Task Completion Delay: The total task completion delay of WD n at epoch k is

$$T_n^{\mathrm{tot}}(k) = x_n^{\mathrm{loc}}(k) T_n^{\mathrm{loc}}(k) + \sum_{m\in\mathcal{M}} a_{n,m}(k) T_{n,m}^{\mathrm{uav}}(k), \tag{29}$$

which becomes zero when $\delta_n(k) = 0$ due to (22). The average task completion delay over all generated tasks is

$$\bar{T} = \frac{\sum_{k\in\mathcal{K}} \sum_{n\in\mathcal{N}} \delta_n(k) T_n^{\mathrm{tot}}(k)}{\sum_{k\in\mathcal{K}} \sum_{n\in\mathcal{N}} \delta_n(k)}. \tag{30}$$

F. Energy Consumption Model

The total system energy includes WD local computation, WD uplink transmission, UAV computation, UAV cooperative service-fetching communication, and UAV propulsion energy:

$$E^{\mathrm{total}} = \sum_{n\in\mathcal{N}} \left(E_n^{\mathrm{loc}} + E_n^{\mathrm{tx}}\right) + \sum_{m\in\mathcal{M}} \left(E_m^{\mathrm{cmp}} + E_m^{\mathrm{comm}} + E_m^{\mathrm{fly}}\right). \tag{31}$$

1) WD Energy Consumption: Following the dynamic voltage and frequency scaling (DVFS) model, the local computation energy of WD n is

$$E_n^{\mathrm{loc}} = \sum_{k\in\mathcal{K}} x_n^{\mathrm{loc}}(k) \delta_n(k) \kappa_n \left(f_n^{\mathrm{loc}}\right)^2 F_n(k), \tag{32}$$

where κ_n and f_n^{loc} denote the effective switched capacitance and local CPU frequency, respectively. The uplink transmission energy is

$$E_n^{\mathrm{tx}} = \sum_{k\in\mathcal{K}} \sum_{m\in\mathcal{M}} a_{n,m}(k) \delta_n(k) \frac{p_n(k) D_n(k)}{R_{n,m}(k)}. \tag{33}$$

2) UAV Energy Consumption: The energy consumed by UAV m is

$$E_m^{\mathrm{total}} = E_m^{\mathrm{cmp}} + E_m^{\mathrm{comm}} + E_m^{\mathrm{fly}}. \tag{34}$$

The onboard computation energy is

$$E_m^{\mathrm{cmp}} = \sum_{k\in\mathcal{K}} \sum_{n\in\mathcal{N}} a_{n,m}(k) \delta_n(k) \kappa_m \left(f_m^{\mathrm{uav}}\right)^2 F_n(k), \tag{35}$$

where κ_m and f_m^{uav} are the UAV capacitance coefficient and CPU frequency.

Let $y_{n,m,m'}(k) \in \{0,1\}$ indicate whether UAV m' sends the required service of task $T_n(k)$ to serving UAV m. The cooperative service-fetching communication energy of UAV m' is

$$E_{m'}^{\mathrm{comm}} = \sum_{k\in\mathcal{K}} \sum_{n\in\mathcal{N}} \sum_{\substack{m\in\mathcal{M} \\ m\neq m'}} y_{n,m,m'}(k) P_{m'}^{\mathrm{uav}} T_{n,m}^{\mathrm{fetch}}(k). \tag{36}$$

The BS is assumed grid-powered; hence, its service-retrieval energy is not included in the UAV energy budget.

For rotary-wing UAVs, the propulsion power at speed $v_m(k)$ is modeled as [24]

$$P_m^{\mathrm{fly}}(k) = \eta_1 \left(1 + \frac{3v_m^2(k)}{v_{\mathrm{tip}}^2}\right) + \eta_2 \left(\sqrt{\eta_3 + \frac{v_m^4(k)}{4}} - \frac{v_m^2(k)}{2}\right)^{\frac{1}{2}} + \eta_4 v_m^3(k), \tag{37}$$

where v_{tip} and $\eta_1, \eta_2, \eta_3, \eta_4$ are aerodynamic constants, and $v_m(k) = \frac{\|\mathbf{q}_m(k+1) - \mathbf{q}_m(k)\|_2}{\Delta t}$. The total propulsion energy is

$$E_m^{\mathrm{fly}} = \Delta t \sum_{k=1}^{K-1} P_m^{\mathrm{fly}}(k). \tag{38}$$

Finally, the UAV energy budget is

$$E_m^{\mathrm{cmp}} + E_m^{\mathrm{comm}} + E_m^{\mathrm{fly}} \le E_m^{\mathrm{max}}, \quad \forall m \in \mathcal{M}. \tag{39}$$

G. Problem Formulation

The objective is to minimize the average task completion latency while regularizing UAV energy consumption through joint optimization of UAV trajectory, task offloading, transmission power, and service caching decisions. Due to binary offloading/caching variables, continuous trajectory/power variables, and nonlinear coupling among communication, computation, service fetching, and energy models, the problem is formulated as a stochastic mixed-integer non-linear programming (MINLP). The optimization problem is formulated as follows:

$$\min_{X} \ \frac{1}{\sum_{k\in\mathcal{K}} \sum_{n\in\mathcal{N}} \delta_n(k)} \sum_{k\in\mathcal{K}} \sum_{n\in\mathcal{N}} \delta_n(k)\, T_n^{\mathrm{tot}}(k) + \mu \sum_{k\in\mathcal{K}} \sum_{m\in\mathcal{M}} E_m^{\mathrm{total}}(k) \tag{40}$$

where $\mu \ge 0$ is a non-negative weighting coefficient that controls the trade-off between latency and energy consumption, $X = \{\mathbf{q}_m(k),\, a_{n,m}(k),\, x_n^{\mathrm{loc}}(k),\, p_n(k),\, c_{n,l}(k),\, c_{m,l}(k)\}$ represents the set of optimization variables, and $T_n^{\mathrm{tot}}(k)$ denotes the total task completion delay. The total UAV energy consumption is given by $E_m^{\mathrm{total}}(k) = E_m^{\mathrm{cmp}}(k) + E_m^{\mathrm{comm}}(k) + E_m^{\mathrm{fly}}(k)$.

$$\mathcal{C}_1: \quad x_n^{\mathrm{loc}}(k) + \sum_{m\in\mathcal{M}} a_{n,m}(k) = \delta_n(k), \qquad \forall n,k \tag{41}$$

$$\mathcal{C}_2: \quad x_n^{\mathrm{loc}}(k) \le c_{n,s_n(k)}(k), \qquad \forall n,k \tag{42}$$

$$\mathcal{C}_3: \quad \sum_{l\in\mathcal{L}} c_{n,l}(k)\, s_l \le C_n, \qquad \forall n,k \tag{43}$$

$$\mathcal{C}_4: \quad \sum_{l\in\mathcal{L}} c_{m,l}(k)\, s_l \le C_m^{\mathrm{uav}}, \qquad \forall m,k \tag{44}$$

$$\mathcal{C}_5: \quad 0 \le p_n(k) \le p_n^{\mathrm{max}}, \qquad \forall n,k \tag{45}$$

$$\mathcal{C}_6: \quad \|\mathbf{q}_m(k+1) - \mathbf{q}_m(k)\| \le v_m^{\mathrm{max}} \Delta t, \qquad \forall m,k \tag{46}$$

$$\mathcal{C}_7: \quad h_{\min} \le h_m(k) \le h_{\max}, \qquad \forall m,k \tag{47}$$

$$\mathcal{C}_8: \quad E_m^{\mathrm{cmp}} + E_m^{\mathrm{comm}} + E_m^{\mathrm{fly}} \le E_m^{\mathrm{max}}, \qquad \forall m \tag{48}$$

$$\mathcal{C}_9: \quad \mathbf{q}_m(k) \in \mathcal{A}, \qquad \forall m,k \tag{49}$$

$$\mathcal{C}_{10}: \quad |\mathcal{N}_m(k)| \le N_m^{\mathrm{max}}, \qquad \forall m,k \tag{50}$$

$$\mathcal{C}_{11}: \quad R_{n,m}(k) \ge a_{n,m}(k)\, R_n^{\mathrm{min}}, \qquad \forall n,m,k \tag{51}$$

Constraint $\mathcal{C}_1$ enforces unique task assignment, while $\mathcal{C}_2$ ensures WD-cache feasibility for local execution. Since the BS stores the complete service library, UAV cache misses affect $T_{n,m}^{\mathrm{fetch}}(k)$ rather than blocking UAV offloading. Constraints $\mathcal{C}_3$–$\mathcal{C}_5$ impose cache and transmit-power limits. Constraints $\mathcal{C}_6$–$\mathcal{C}_9$ define UAV mobility, altitude, energy, and deployment-region constraints. Finally, $\mathcal{C}_{10}$ and $\mathcal{C}_{11}$ enforce NOMA cluster-size and QoS requirements, respectively.

IV. Proposed Methodology

A. Overall Framework for Joint Optimization

The formulated problem involves hybrid decision variables, stochastic system dynamics, and strong coupling among task

offloading, cooperative caching, NOMA-based resource allocation, UAV trajectory, and jammer-induced interference. These couplings make the stochastic MINLP difficult to solve directly using conventional optimization methods. In particular, caching affects service availability and service-fetching delay, offloading determines traffic distribution and NOMA interference, and UAV mobility changes channel quality, jamming exposure, and propulsion energy consumption. To address these challenges, we propose the *Diffusion-based Multi-UAV Joint Optimization (DMJO)* framework. DMJO decomposes the original problem into four tightly coupled modules: (i) adaptive cooperative caching for service placement, (ii) MAPPO-based UAV trajectory control, (iii) diffusion-enhanced DRL for combinatorial task offloading, and (iv) analytical NOMA-based power allocation. These modules are jointly coordinated through a hierarchical learning framework to minimize task completion latency while regularizing UAV energy consumption.

To capture the different update rates of physical mobility, caching, and offloading decisions, DMJO adopts a *two-timescale hierarchical framework*. The mission horizon is divided into macro-intervals and micro-epochs:

- **Macro-timescale:** UAV trajectory and cooperative caching decisions are updated periodically and kept fixed within each macro-interval, since mobility adjustment and service replacement introduce physical and communication overhead.
- **Micro-timescale:** Given the macro-level trajectory and caching states, diffusion-enhanced task offloading and analytical NOMA power allocation are performed at each epoch to respond to stochastic task arrivals, time-varying channels, and jammer interference.

The learning modules operate under a CTDE paradigm. During offline training, a centralized critic uses global system information to guide policy learning, while during online execution, each UAV makes real-time decisions using only local observations and the learned policies. This design enables scalable multi-UAV coordination in dynamic jammer-aware MEC-IoT networks.

B. Hierarchical MDP Formulation

To solve the stochastic MINLP, we model the system as a Hierarchical Markov Decision Process (HMDP). The system operates over macro-intervals indexed by t with duration $\Delta T_c = K_c \Delta t$, where UAV trajectory and caching decisions are updated, and micro-epochs indexed by k, where task offloading and communication decisions are performed.

1) State Space and Local Observations:

$$\begin{aligned}\mathcal{S}(k) = \{&\mathbf{w}_n(k), \mathbf{q}_m(k), g_{n,m}(k), I_m(k), Q_m(k), c_{n,l}(k),\\ &c_{m,l}(k), D_n(k), F_n(k), s_n(k), \delta_n(k)\}, \quad \forall n, m, l.\end{aligned} \tag{52}$$

This state captures network geometry, channel conditions, queue states, caching states, and task characteristics. During centralized training, the critic observes $\mathcal{S}(k)$, whereas during decentralized execution, UAV m acts based on its local observation $o_m(tK_c) \subset \mathcal{S}(tK_c)$, including local channels, queue status, residual energy, and relative distances to nearby WDs and jammers.

2) Hierarchical Action Space: The learning-based action space is divided according to the decision timescale. At the macro-timescale, MAPPO outputs the UAV velocity actions:

$$\mathcal{A}_{\text{macro}}(t) = \{\mathbf{v}_m(tK_c)\}_{m\in\mathcal{M}}, \tag{53}$$

where $\mathbf{v}_m(tK_c) = [v_m^x(tK_c), v_m^y(tK_c), v_m^h(tK_c)]^\top$ controls horizontal and vertical UAV movement.

At the micro-timescale, the diffusion-enhanced actor generates offloading decisions:

$$\mathcal{A}_{\text{micro}}(k) = \{a_{n,m}(k), x_n^{\text{loc}}(k)\}_{n\in\mathcal{N},m\in\mathcal{M}}. \tag{54}$$

The caching states $\{c_{m,l}(tK_c)\}$ are updated by the adaptive caching module, while the transmit powers are obtained analytically through NOMA-based optimization and are not included in the RL action space.

3) State Transition: The transition from $\mathcal{S}(k)$ to $\mathcal{S}(k+1)$ follows the selected actions and physical dynamics. WD mobility follows the Gauss-Markov model, while UAV positions evolve at the macro-timescale as

$$\mathbf{q}_m(k + K_c) = \mathbf{q}_m(k) + \mathbf{v}_m(tK_c)\Delta T_c. \tag{55}$$

The updated UAV positions affect channel gains, jammer interference, service coverage, and queue evolution, while caching states remain fixed within each macro-interval.

4) Reward Function: To align with (40), the reward is defined as

$$r(k) = \tfrac{1}{2}\, r^{\text{global}}(k) + \tfrac{1}{2} \sum_{m\in\mathcal{M}} r_m^{\text{shaped}}(k), \tag{56}$$

where

$$r^{\text{global}}(k) = -\alpha_L \left(\bar{T}(k) - \frac{N}{2M}\right) - \frac{\beta_E}{10^3} \sum_{m\in\mathcal{M}} E_m^{\text{total}}(k), \tag{57}$$

with $(\alpha_L, \beta_E)=(0.6, 0.4)$ from (40). The shaping term $r_m^{\text{shaped}}(k)$ adds per-task outcome rewards and penalises UAV collisions and boundary violations.

5) Objective: The objective of the HMDP is to learn an optimal joint policy $\pi^* = \{\pi_\phi^*, \pi_{\theta_m}^*\}$ that maximizes the expected cumulative discounted reward:

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{\tau\sim\pi}\left[\sum_{k=1}^{K} \gamma^{k-1} r(k)\right], \tag{58}$$

where $\gamma \in (0, 1]$ is the discount factor and τ denotes the trajectory of states and actions.

C. Cooperative Caching Module

The caching module updates service placement at the macro-timescale to improve UAV-level service availability and reduce service retrieval delay. As defined in Section III-D, $c_{n,l}(k)$ and $c_{m,l}(k)$ denote WD and UAV cache states, respectively, while $\tilde{c}_{m,l}^{\text{uav}}(k)$ captures whether service l is available at UAV m either locally or through A2A cooperative fetching. BS-assisted retrieval guarantees system-level service availability but introduces additional fetching delay.

1) Adaptive Cooperative Caching Objective: Each service l is assigned a priority score based on request frequency and successful task completion:

$$S_l(k) = \lambda_f \frac{N_l(k)}{\sum_{l' \in \mathcal{L}} N_{l'}(k) + \varepsilon} + \lambda_w \frac{W_l(k)}{\sum_{l' \in \mathcal{L}} W_{l'}(k) + \varepsilon}, \quad (59)$$

where $N_l(k)$ is the number of requests for service l, $W_l(k)$ is the number of successfully completed tasks using service l, and $\lambda_f + \lambda_w = 1$. The cooperative caching utility is

$$U^{\text{cache}}(k) = \sum_{n \in \mathcal{N}} \sum_{l \in \mathcal{L}} S_l(k) c_{n,l}(k) + \sum_{m \in \mathcal{M}} \sum_{l \in \mathcal{L}} S_l(k) \tilde{c}^{\text{uav}}_{m,l}(k). \quad (60)$$

Maximizing $U^{\text{cache}}(k)$ increases the availability of high-impact services at WDs and cooperative UAV caches, thereby reducing fetching delay.

2) Caching Update Strategy and Timescale Separation: Since cache replacement incurs service-transfer overhead, UAV caches are updated only at the beginning of each macro-interval and remain fixed within that interval:

$$c_{m,l}(k+1) = \begin{cases} 1, & \text{if } l \in \mathcal{L}^{\text{opt}}_m(k) \text{ and } k \mod K_c = 0, \\ c_{m,l}(k), & \text{otherwise.} \end{cases} \quad (61)$$

The set $\mathcal{L}^{\text{opt}}_m(k)$ is obtained through a greedy knapsack strategy, where services are sorted by $S_l(k)$ and selected until the UAV cache capacity is exhausted. A diversity-aware placement strategy is used to reduce redundant caching across UAVs.

D. NOMA-Based Resource Allocation Module

Given UAV positions and offloading decisions, uplink power-domain NOMA allows multiple WDs to transmit simultaneously to the same UAV. The SINR $\gamma_{n,m}(k)$ and achievable rate $R_{n,m}(k)$ follow (14) and (15), and depend on UAV geometry, intra-cluster interference, and jammer interference. For a fixed NOMA cluster and SIC decoding order, the WD transmit powers $\{p_n(k)\}$ are optimized analytically rather than learned by RL. Specifically, the power allocation minimizes transmission delay subject to QoS and power constraints:

$$\begin{aligned} \min_{\{p_n(k)\}} \quad & \sum_{n \in \mathcal{N}_m(k)} \frac{D_n(k)}{R_{n,m}(k)} \\ \text{s.t.} \quad & R_{n,m}(k) \geq R^{\min}_n, \quad \forall n \in \mathcal{N}_m(k), \\ & 0 \leq p_n(k) \leq p^{\max}_n, \quad \forall n \in \mathcal{N}_m(k). \end{aligned} \quad (62)$$

This subproblem is solved using bisection-based feasibility search. By decoupling transmit power from the RL action space, the proposed design reduces policy dimensionality and improves learning stability.

E. Diffusion-Enhanced UAV Control Module

Standard Gaussian MLP policies explore the continuous macro-timescale UAV control space with unimodal distributions. To improve structured exploration under stochastic channels, wind, and jamming, we employ a dual-head diffusion-enhanced actor.

1) Per-Agent Action Representation: Each UAV m outputs a 4-dimensional continuous action $\mathbf{a}_m(k) \in [-1, 1]^4$, where a^x_m, a^y_m are normalised horizontal velocity components, a^h_m is an altitude-change command (applied at macro boundaries, clamped to $[h_{\min}, h_{\max}]$), and a^s_m is a scheduling priority hint for NOMA cluster selection. UAV position evolves as $\mathbf{q}_m(k+K_c) = \mathbf{q}_m(k) + [a^x_m v^{\max}_m,\, a^y_m v^{\max}_m,\, a^h_m \Delta h^{\max}]^\top \Delta T_c$, with momentum blending $\tilde{v}_m = 0.65 \tilde{v}^{\text{prev}}_m + 0.35\, a^{x/y}_m v^{\max}_m$ applied to smooth trajectories.

2) Dual-Head Diffusion Actor: A shared encoder f_ψ produces features $\mathbf{z}_m = f_\psi(o_m)$ fed to two heads.

Gaussian head: Parameterises $\mathcal{N}(\boldsymbol{\mu}_\theta, \text{diag}(\boldsymbol{\sigma}^2_\theta))$; actions are drawn as $\tilde{\mathbf{a}}_m = \tanh(\mathbf{u})$ with log-probability corrected for $\tanh$ squashing:

$$\log \pi_\theta(\mathbf{a}_m | \mathbf{z}_m) = \textstyle\sum_i \big[\log \mathcal{N}(u_i; \mu_i, \sigma^2_i) - \log(1 - a^2_{m,i} + \varepsilon)\big]. \quad (63)$$

Diffusion head: Uses a VP-SDE noise schedule with cosine-shaped betas $\beta_k = 1 - \exp\big(-\frac{\beta_a}{K} - \frac{2k-1}{2K^2}(\beta_b - \beta_a)\big)$, cumulative products $\bar{\alpha}_k = \prod_{i=1}^{k}(1 - \beta_i)$, and posterior variance $\tilde{\beta}_k = \frac{1-\bar{\alpha}_{k-1}}{1-\bar{\alpha}_k}\beta_k$. Starting from $\mathbf{a}^{(K)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, the reverse process iterates for $k = K, \ldots, 1$:

$$\mathbf{a}^{(k-1)}_m = \frac{1}{\sqrt{1-\beta_k}} \left(\mathbf{a}^{(k)}_m - \frac{\beta_k}{\sqrt{1-\bar{\alpha}_k}} \epsilon_\phi(\mathbf{a}^{(k)}_m, k, \mathbf{z}_m) \right) + \sqrt{\tilde{\beta}_k}\, \boldsymbol{\xi}_k, \quad (64)$$

where $\boldsymbol{\xi}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ for $k > 1$ and $\boldsymbol{\xi}_1 = \mathbf{0}$; the final action is $\mathbf{a}^{\text{diff}}_m = \tanh(\mathbf{a}^{(0)}_m)$.

3) Blend Annealing and Joint Training: During rollout, the diffusion action is selected with probability $\rho_{\text{blend}}(p) = \max(0.3,\, 0.7 - 0.4p)$, where $p \in [0, 1]$ is training progress, annealing diffusion dominance from 70% to 30% as the Gaussian head matures. Since the diffusion head lacks a tractable likelihood, all rollout actions are scored by the Gaussian head via (63) to form consistent PPO importance ratios. The joint actor loss is

$$\mathcal{L}_{\text{actor}} = -\mathcal{L}_{\text{PPO}} - c_{\text{ent}} \mathcal{L}_{\text{ent}} + \lambda_d\, \mathbb{E}_{k, \boldsymbol{\epsilon}} \big[\|\boldsymbol{\epsilon} - \epsilon_\phi(\mathbf{a}^{(k)}_m, k, \mathbf{z}_m)\|^2 \big], \quad (65)$$

where $\lambda_d = 0.1$ and $K = 3$ denoising steps suffice for real-time inference; experiments showed no significant gain beyond $K = 5$ in this low-dimensional action space, consistent with [22].

F. MAPPO-Based UAV Trajectory Optimization Module

UAV trajectory directly affects channel quality, jammer exposure, service coverage, and propulsion energy. Therefore, we employ MAPPO under the CTDE paradigm, where a centralized critic uses the global state during training, while each UAV executes the shared policy using its local observation $o_m(k)$. The shared policy improves sample efficiency, whereas agent-specific observations capture UAV heterogeneity such as position, capability, and local interference conditions. Updated UAV positions influence channel gains $g_{n,m}(k)$, jammer interference $I_m(k)$, service coverage, and propulsion energy. Wind-induced perturbations are captured through the state representation, enabling the learned policy to adapt trajectory decisions under environmental disturbances.

1) MAPPO Optimization: All UAVs share the cooperative reward $r(k)$ in (56). During centralized training, a critic $V_\omega(\mathcal{S}(k))$ estimates the global value function, and the shared policy is updated using the clipped PPO objective:

$$\mathcal{L}^{\mathrm{MAPPO}}(\theta) = \mathbb{E}[\min(\rho(k)A(k), \mathrm{clip}(\rho(k), 1-\epsilon, 1+\epsilon)A(k))], \tag{66}$$

where $\rho(k)$ is the importance sampling ratio and $A(k)$ is computed using GAE.

G. Joint Optimization Algorithm and Complexity Analysis

The proposed DMJO framework integrates adaptive cooperative caching, NOMA-based communication, diffusion-enhanced task offloading, and MAPPO-based UAV trajectory optimization under the CTDE paradigm. The optimization consists of offline centralized training and online decentralized execution.

1) Offline Centralized Training Algorithm: During offline training, a centralized server simulates the environment and jointly updates the diffusion-enhanced actor-critic networks and MAPPO trajectory policies over N_e episodes. The training follows the proposed two-timescale structure, where trajectory/caching decisions are updated at the macro-level and offloading/communication decisions are updated at the micro-level. The overall procedure is summarized in Algorithm 1.

2) Online Decentralized Execution and Complexity: During online execution, each UAV uses its local observation $o_m(tK_c)$ to compute trajectory actions, while the diffusion head generates offloading decisions over T_d reverse denoising steps, followed by analytical NOMA power allocation. The offline training complexity is dominated by diffusion sampling and policy updates, and scales as $\mathcal{O}(N_e K\,(T_d|\phi| + |\phi| + |\theta| + M|\theta_m|))$. During online execution, adaptive caching incurs $\mathcal{O}(ML\log L)$ per macro-interval, diffusion-based offloading requires $\mathcal{O}(T_d|\phi|)$ per micro-epoch, and NOMA bisection incurs $\mathcal{O}(1)$ per user pair. Thus, the per-epoch online complexity is $\mathcal{O}\Big(T_d|\phi| + N + \frac{1}{K_c}\,(M|\theta_m| + ML\log L)\Big)$, which supports real-time decentralized implementation.

V. Simulation Results

1) Simulation Setup: The proposed Diffusion-MAPPO framework is implemented in Python using PyTorch ≥ 2.0 on a single NVIDIA RTX A4000 GPU (16 GB VRAM), trained for 5×10^6 environment steps with domain randomisation every 100k steps, converging in approximately **108 hours**. The source code is hosted at https://github.com/Khusu0511/Diffusion_DRL_MultiUAV-DMJO and is available upon reasonable request to the corresponding author. Detailed simulation parameters are listed in Table I.

2) Benchmark Comparison: To evaluate the effectiveness of the proposed DMJO framework, we compare it with representative learning-based and heuristic baselines, including MAPPO [3], MADDPG [26], MATD3 [27], greedy offloading, and local execution. MAPPO evaluates the impact of diffusion-enhanced offloading by removing the diffusion head, while MADDPG and MATD3 serve as deterministic multi-agent DRL baselines. Greedy offloading assigns each task to the nearest UAV without caching awareness, whereas local execution processes all tasks at the originating WDs.

3) Convergence Analysis: Fig. 2 shows the convergence behavior of Diff-MAPPO under different actor learning rates. The proposed setting $\alpha_a = 3\times10^{-4}$ achieves the best balance between convergence speed and stability, while smaller and

Algorithm 1 Proposed DMJO Framework

Require: Number of WDs N, UAVs M, horizon K, macro-interval K_c, diffusion steps T_d, episodes N_e
Ensure: Trained parameters ϕ (Diffusion Head), ω (Centralized Critic), θ (Shared MAPPO Actor)

```
Initialize φ, ω and θ
Initialize trajectory buffer D
for episode e = 1 to N_e do
    Reset environment and initialize UAV positions {q_m(0)}
    for macro-interval t = 0 to ⌊K/K_c⌋ − 1 do
        Macro-Step: Adaptive Caching and Trajectory Update
        Update cooperative caching using priority scores S_l(k)
        for each UAV m do
            Sample velocity action a_m(tK_c) ~ π_θ(· | o_m(tK_c))
            Update UAV position q_m((t+1)K_c) ← q_m(tK_c) + v_m(tK_c)ΔT_c
        end for
        for micro-epoch k = tK_c to (t+1)K_c − 1 do
            Micro-Step: Offloading and Communication
            Observe global state S(k)
            Denoise ã_k for T_d steps
            Project ã_k to discrete offloading decisions
            Compute NOMA powers {p_n^*(k)} via bisection
            Compute reward r(k) based on delay and energy
            Store transition (S(k), A(k), r(k), S(k+1)) in D
        end for
    end for
    Parameter Update
    Update φ using noise prediction loss and policy gradient
    Update ω using TD-error minimization
    Update θ using PPO objective with GAE
end for
Return: φ, ω, θ
```

Table I: Simulation Parameters

Parameter	Value	Parameter	Value
Network and Communication			
Area radius (R)	500 m	WDs ($\mathcal{N}$)	100 (80–150)
UAVs ($\mathcal{M}$)	5 (3–8)	Jammers ($\mathcal{J}$)	1 (1–3)
UAV CPU f_m^{uav}	1.2–3.2 GHz	Altitude (h_m)	85–115 m
UAV cache	60–160 units	Cached programs	6–15
Bandwidth (B_m)	8–16 MHz	Sub-channels	5
UAV Tx power	22–28 dBm	BS bandwidth	10 MHz
WD Tx power	20 dBm	WD Tx range	15–23 dBm
Jammer Tx power	20 dBm	Path loss exp.	2.5
Propulsion, Wind, and Caching			
P_1, P_2	79.86, 88.63 W	U_{tip}, v_0	120, 4.03 m/s
d_0, ρ, s, A_r	0.6, 1.225, 0.05, 0.503	Wind speed	2.5 m/s
Wind range	0–8 m/s	θ, σ_w	0.08, 0.4
Drag coeff.	0.05 N s/m	Quad. drag coeff.	0.012 W s^2/m^2
WD drift factor	0.12	Dir. change rate	0.1 rad/slot
Zipf exponent (ζ)	1.0	Service programs	20
Program size	1–5 units	Cache update intv.	25 slots
A2A bandwidth	5 MHz	Coop. fetch delay	0.05 s
Task, Energy, and Training			
WD CPU freq.	0.5 GHz	Eff. capacitance (κ)	10^{-28}
Task data size	0.5–5.0 Mbit	Task CPU cycles	0.1–1.0 Gcyc.
Task deadline	0.2–2.0 s	Task gen. prob.	0.7
Task types	8	WD energy buffer	1.0 J
Episode length	500 slots	Slot duration (Δt)	1.0 s
Denoising steps	3	Hidden dimension	256
Actor/Critic LR	$3\times10^{-4}/10^{-4}$	Discount/GAE	0.99/0.95
PPO clip	0.2	Total timesteps	5×10^6
Optimizer/Activation	Adam/ReLU	Mini-batches	5

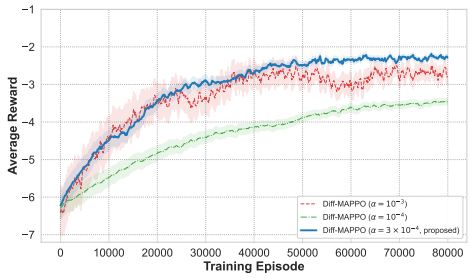
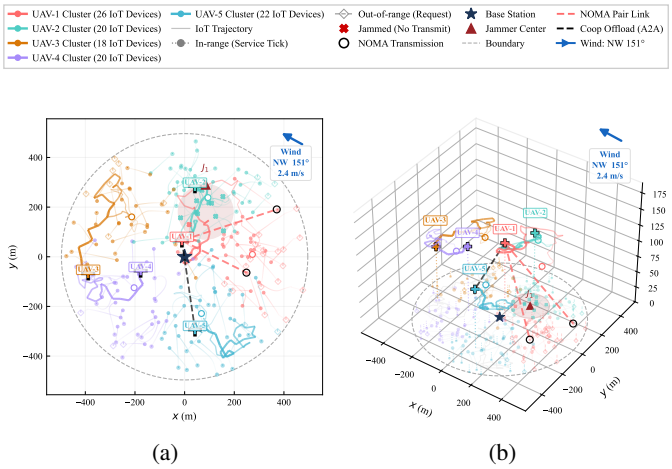


Fig. 2. Convergence behavior of DMJO under different actor learning rates (α_a).

larger learning rates lead to slower convergence and stronger reward fluctuations, respectively. This confirms that the selected learning rate provides stable exploration and policy improvement.



(a) (b)

Fig. 3. Trajectory evolution of multiple UAVs under wind and jammer-aware communication conditions: (a) 2D UAV trajectory plot and (b) 3D UAV trajectory plot.

4) UAV Trajectory Analysis: Fig. 3 shows the learned UAV trajectories under jammer interference and NW 151° wind at 2.4 m/s. The 2D trajectory 3a shows that UAVs move toward high-demand WD clusters while maintaining about 30-50 m separation from jammer-affected regions. The 3D 3b view further shows altitude adaptation, where UAVs increase altitude in high-interference regions to improve LoS connectivity and lower altitude near dense WD clusters to reduce path loss. These results confirm jammer-aware and wind-robust mobility control.

5) Latency Performance Analysis: Fig. 4 compares the average task completion delay under varying WD density, UAV deployment size, and jammer intensity. As shown in Fig. 4a, delay increases with the number of WDs due to higher task load and resource contention. In Fig. 4b, adding more UAVs reduces delay by increasing computation availability and shortening WD-UAV transmission distances. Fig. 4c shows that delay increases with jammer intensity because of stronger uplink interference. Across all cases, Diff-MAPPO achieves the lowest delay by jointly optimizing offloading, caching, communication, and UAV mobility, while adapting UAV positions and offloading decisions to mitigate jamming effects.

6) Energy Consumption Analysis: Fig. 5 presents the energy performance under varying UAV deployment size, task size, and WD density. As shown in Fig. 5a, total energy consumption increases with the number of UAVs due to higher propulsion and communication overhead. Nevertheless, Diff-MAPPO achieves the lowest energy consumption by learning energy-aware trajectories and avoiding unnecessary UAV movement. Fig. 5b reports task processing energy only, excluding propulsion energy and considering transmission, UAV computation, and local computation costs. As task size increases, processing energy rises for all methods due to higher transmission and computation demand, while Diff-MAPPO remains the most energy-efficient through effective local/offloaded task assignment. Fig. 5c shows that total energy consumption increases with WD density because of more task arrivals, transmission demand, and computation load. Overall, Diff-MAPPO consistently consumes less energy than MAPPO, MATD3, MADDPG, greedy offloading, and local execution by balancing workload distribution and reducing redundant energy expenditure.

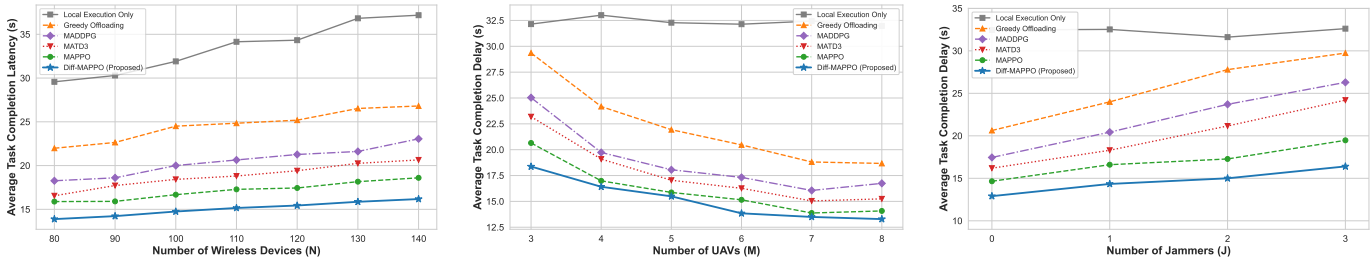
7) Impact of Cache Capacity and Service Popularity: Fig. 6 evaluates caching performance under different cache capacities and Zipf popularity parameters. As shown in Fig. 6a, increasing normalized cache capacity reduces task completion delay because more services are served from local or cooperative UAV caches instead of BS-assisted fetching. Diff-MAPPO achieves the lowest delay, providing about 10–15% improvement over MAPPO. Fig. 6b shows that cache hit rate increases with the Zipf parameter, since requests become more concentrated on popular services. Diff-MAPPO reaches nearly 90% cache hit rate at higher Zipf values, confirming the effectiveness of adaptive cooperative caching in improving UAV-level service availability and reducing retrieval latency. The learned caching policy serves 17.4% of requests through local cache hits, 75.0% through cooperative UAV cache hits, and only 7.6% through BS-assisted fetching, showing reduced dependence on the BS repository.

8) Ablation Study: To evaluate the contribution of each component in the proposed DMJO framework, we conduct an ablation study by selectively disabling key modules. The results are summarized in Table II.

Table II: Ablation Study of DMJO

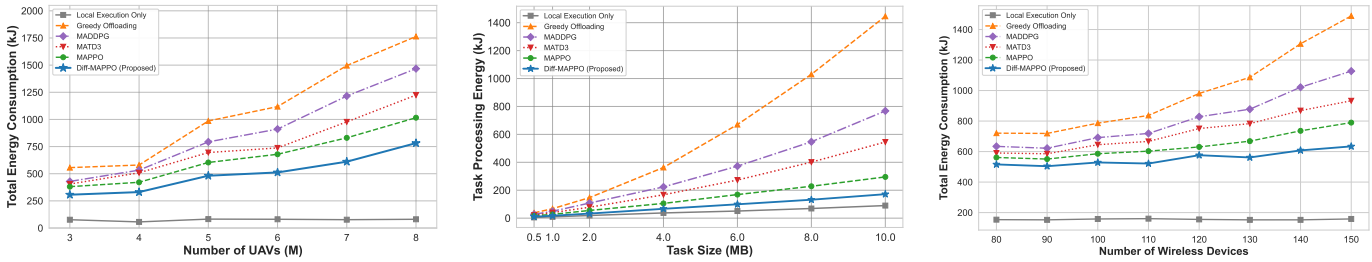
Method	**Latency (s)**	**Energy (10^3 J)**
DMJO (Full)	14.7	483
Single-timescale DMJO	16.3	667
w/o Diffusion	15.9	512
w/o MAPPO	19.1	715
w/o Cooperative Caching	17.6	613
w/o Diffusion + Cache	19.7	734

The ablation results show that the full DMJO framework achieves the lowest latency and energy consumption. Removing two-timescale control increases energy usage, while removing diffusion weakens offloading efficiency. The largest degradation occurs without MAPPO-based trajectory optimization, confirming the importance of adaptive UAV mobility. Overall, each module to the final performance gain.



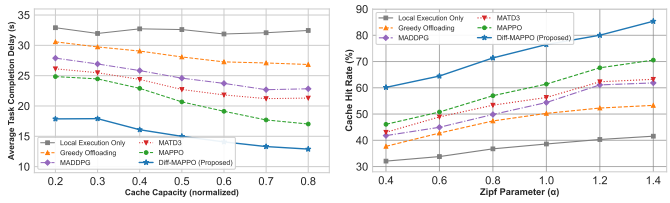
(a) (b) (c)

Fig. 4. Average task completion delay under different network conditions: (a) impact of the number of WDs, (b) impact of the number of UAVs, and (c) impact of the number of jammers.



(a) (b) (c)

Fig. 5. Energy performance comparison under different system conditions: (a) total energy consumption versus number of UAVs, (b) task processing energy versus task size, and (c) total energy consumption versus number of WDs.



(a) (b)

Fig. 6. Caching performance analysis: (a) average task completion delay versus normalized cache capacity and (b) cache hit rate versus Zipf popularity parameter.

VI. Conclusion and Future Work

In this paper, we proposed DMJO, a diffusion-based multi-UAV joint optimization framework for jammer-aware MEC-IoT networks. The proposed framework jointly optimizes task offloading, cooperative 3-tier service availability, NOMA resource allocation, and UAV trajectory control under dynamic task arrivals and adversarial interference. A two-timescale hierarchical MDP was developed to separate slow-varying trajectory/caching decisions from fast-timescale offloading and communication decisions. To improve exploration in the combinatorial offloading space, a dual-head diffusion-enhanced actor was integrated with PPO-based policy learning, while NOMA transmit powers were obtained through analytical bisection-based optimization. Simulation results demonstrate that DMJO achieves lower task completion delay and energy consumption than benchmark DRL and heuristic methods, while maintaining robustness under jamming and wind disturbances. Future work will extend the framework toward online adaptation under unknown jammer strategies and real-world UAV deployment constraints.

References

[1] Y. Yin, M. Bouazizi, B. Hu, G. Gui, and T. Ohtsuki, "Integrated Deployment and Resource Allocation in Multilayer UAV-Enabled NOMA Wireless Caching Networks," *IEEE Internet of Things Journal*, vol. 13, no. 7, pp. 14 898–14 911, 2026.

[2] N. Talukdar, A. Raghav, A. Hazra, D. C. Barman, and N. Mazumdar, "A Deep Deterministic Policy Gradient Method for Optimizing Task Completion Time and Energy Efficiency in UAV-Assisted IoT Networks," *IEEE Internet of Things Journal*, vol. 12, no. 15, pp. 31 907–31 917, 2025.

[3] H. Kang, X. Chang, J. Mišić, V. B. Mišić, J. Fan, and Y. Liu, "Cooperative UAV Resource Allocation and Task Offloading in Hierarchical Aerial Computing Systems: A MAPPO-Based Approach," *IEEE Internet of Things Journal*, vol. 10, no. 12, pp. 10 497–10 509, 2023.

[4] Y. Chen, Y. Yang, Y. Wu, J. Huang, and L. Zhao, "Joint Trajectory Optimization and Resource Allocation in UAV-MEC Systems: A Lyapunov-Assisted DRL Approach," *IEEE Transactions on Services Computing*, vol. 18, no. 2, pp. 854–867, 2025.

[5] L. Yang, J. Zheng, and Y. Zhang, "A Deep Reinforcement Learning-Based Service Caching and Computation Offloading Algorithm for Multi-UAV-Enabled MEC Systems," *IEEE Transactions on Cognitive Communications and Networking*, vol. 12, pp. 6658–6669, 2026.

[6] M. N. Tariq, J. Wang, S. Raza, M. Siraj, M. Altamimi, and S. Memon, "Toward Optimal Resource Allocation: A Multi-Agent DRL Based Task Offloading Approach in Multi-UAV-Assisted MEC Networks," *IEEE Access*, vol. 12, pp. 81 428–81 440, 2024.

[7] Z. Sun, G. Sun, Q. Wu, L. He, S. Liang, H. Pan, D. Niyato, C. Yuen, and V. C. M. Leung, "TJCCT: A Two-Timescale Approach for UAV-Assisted Mobile Edge Computing," *IEEE Transactions on Mobile Computing*, vol. 24, no. 4, pp. 3130–3147, 2025.

[8] D. C. Barman, S. Chakraborty, A. Hazra, and N. Mazumdar, "Joint Service Caching, Task Offloading, and Service Delivery in Mobile Edge Computing for Latency Critical UAV Networks," *IEEE Transactions on Computational Social Systems*, pp. 1–13, 2026.

[9] Q. Xu, M. Cheng, J. Jin, Z. Su, D. Fang, Y. Wang, and Y. Wu, "Cooperative Edge Content Caching With Popularity Prediction in UAV-Assisted Vehicular Networks," *IEEE Transactions on Network Science and Engineering*, vol. 13, pp. 7614–7632, 2026.

[10] P. Qin, Y. Fu, J. Zhang, S. Geng, J. Liu, and X. Zhao, "DRL-Based Resource Allocation and Trajectory Planning for NOMA-Enabled Multi-UAV Collaborative Caching 6G Network," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 6, pp. 8750–8764, 2024.

[11] L. X. Nguyen, Y. K. Tun, T. N. Dang, Y. M. Park, Z. Han, and C. S. Hong, "Dependency Tasks Offloading and Communication Resource Allocation in Collaborative UAV Networks: A Metaheuristic Approach," *IEEE Internet of Things Journal*, vol. 10, no. 10, pp. 9062–9076, 2023.

[12] S. Araf, A. S. Saha, S. H. Kazi, N. H. Tran, and M. G. R. Alam, "UAV Assisted Cooperative Caching on Network Edge Using Multi-Agent Actor-Critic Reinforcement Learning," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 2, pp. 2322–2337, 2023.

[13] S. Dhara, H. T. Malazi, S. R. Chaudhry, A. Kazmi, and S. Clarke, "Co-Operative Caching for Real-Time Content Retrieval in an Integrated Space-Air-Ground Network for a Post-Disaster Scenario," *IEEE Transactions on Services Computing*, pp. 1–15, 2026.

[14] X. Qi, J. Chong, Q. Zhang, and Z. Yang, "Toward Cooperatively Caching in Multi-UAV-Assisted Network: A Queue-Aware CDS-Based Reinforcement Learning Mechanism With Energy-Efficiency Maximization," *IEEE Internet of Things Journal*, vol. 11, no. 9, pp. 16 461–16 477, 2024.

[15] D. Wang, Y. Bai, G. Huang, B. Song, and F. R. Yu, "Cache-Aided MEC for IoT: Resource Allocation Using Deep Graph Reinforcement Learning," *IEEE Internet of Things Journal*, vol. 10, no. 13, pp. 11 486–11 496, 2023.

[16] B. Tian, L. Wang, L. Xu, W. Pan, H. Wu, L. Li, and Z. Han, "UAV-Assisted Wireless Cooperative Communication and Coded Caching: A Multiagent Two-Timescale DRL Approach," *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 4389–4404, 2024.

[17] A. I. Ameur, O. S. Oubbati, A. Rachedi, A. Arishi, and M. Atiquzzaman, "Intelligent UAV Caching and Energy Management in 6G Networks," *IEEE Transactions on Network Science and Engineering*, vol. 13, pp. 3175–3192, 2026.

[18] P. Qin, M. Fu, Y. Fu, R. Ding, and X. Zhao, "Collaborative Edge Computing and Program Caching With Routing Plan in C-NOMA-Enabled Space-Air-Ground Network," *IEEE Transactions on Wireless Communications*, vol. 23, no. 12, pp. 18 302–18 315, 2024.

[19] H. Wang, G. Ding, J. Chen, Y. Zou, and F. Gao, "UAV Anti-Jamming Communications With Power and Mobility Control," *IEEE Transactions on Wireless Communications*, vol. 22, no. 7, pp. 4729–4744, 2023.

[20] Z. Lv, L. Xiao, Y. Du, G. Niu, C. Xing, and W. Xu, "Multi-Agent Reinforcement Learning Based UAV Swarm Communications Against Jamming," *IEEE Transactions on Wireless Communications*, vol. 22, no. 12, pp. 9063–9075, 2023.

[21] C. Wang, R. Zhang, Z. Xiong, D. Zhai, D. Niyato, and Z. Han, "Diffusion-Based Trajectory and Semantic Resource Optimization in UAV-Assisted Edge Computing," *IEEE Transactions on Wireless Communications*, vol. 25, pp. 11 672–11 687, 2026.

[22] S. Liang, M. Yin, W. Xie, Z. Sun, J. Li, J. Wang, and H. Du, "UAV-Enabled Secure Data Collection and Energy Transfer in IoT via Diffusion-Model-Enhanced Deep Reinforcement Learning," *IEEE Internet of Things Journal*, vol. 12, no. 10, pp. 13 455–13 468, 2025.

[23] L. Liu, H. Miao, C. Qu, Z. Wang, H. Zhang, and Z. Li, "Multi-UAV Path Planning for Mobile Edge Computing With High-Density Mobile Devices," *IEEE Transactions on Mobile Computing*, pp. 1–14, 2026.

[24] N. Talukdar, T. Barai, A. Hazra, and N. Mazumdar, "MAPPO-Driven Task Quality Optimization with Wireless Energy Transfer in Multi-UAV MEC-Enabled IoT Networks," *IEEE Transactions on Sustainable Computing*, pp. 1–14, 2026.

[25] Z. Ma, B. Ai, R. He, G. Wang, Y. Niu, and Z. Zhong, "A Wideband Non-Stationary Air-to-Air Channel Model for UAV Communications," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 2, pp. 1214–1226, 2020.

[26] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and L. Hanzo, "Multi-Agent Deep Reinforcement Learning-Based Trajectory Planning for Multi-UAV Assisted Mobile Edge Computing," *IEEE Transactions on Cognitive Communications and Networking*, vol. 7, no. 1, pp. 73–84, 2021.

[27] H. Pi, X. He, and X. Yang, "MATD3-Based Optimization for UAV Edge Computing Task Offloading," in *2025 5th International Symposium on Artificial Intelligence and Big Data (AIBDF)*, 2025, pp. 1131–1138.



Nayanjit Talukdar received the B.Tech. (IT) degree from the School of Engineering and Technology, Nagaland University, Nagaland, India, and the M.Tech. degree (IT) from Tezpur University, Assam, India. He is currently pursuing the Ph.D. (IT) degree with the Indian Institute of Information Technology, Allahabad, India. His research interests include energy harvesting techniques in wireless sensor networks (WSNs), unmanned aerial vehicles (UAVs), and the Internet of Things (IoT).



Tivranshu received the B.Tech. (CSE) degree from Guru Gobind Singh Indraprastha University, Delhi, India. He is currently pursuing the Integrated Ph.D. (IT) degree at the Indian Institute of Information Technology, Allahabad, India. His research interests include wireless sensor networks, unmanned aerial vehicles, and the Internet of Things.



Kushagra Gupta is currently pursuing the B.Tech. (IT) degree at the Indian Institute of Information Technology, Allahabad, India. His research interests include wireless sensor networks, unmanned aerial vehicles, and the Internet of Things.



Shivam Chaturvedi is currently pursuing the B.Tech. (IT) degree at the Indian Institute of Information Technology, Allahabad, India. His research interests include wireless sensor networks, unmanned aerial vehicles, and the Internet of Things.



Tanmay Mathurvaishya is currently pursuing the B.Tech. (IT) degree at the Indian Institute of Information Technology, Allahabad, India. His research interests include wireless sensor networks, unmanned aerial vehicles, and the Internet of Things.



Nabajyoti Mazumdar (Member, IEEE) received the M.Tech. and Ph.D. degrees from the Indian Institute of Technology (ISM), Dhanbad. He is currently an Assistant Professor with the Department of Information Technology, Indian Institute of Information Technology, Allahabad, India. His research focuses on wireless sensor networks, the Internet of Things (IoT), and soft computing techniques.